# Maîtrise des Structures Répétitives (Boucles), Preuve de Terminaison et Invariants

**Thème principal :** Algorithmique et Programmation - Structures itératives, variants et invariants de boucle

**Synthèse exécutive :** Ce cours pose les bases fondamentales de l'itération en programmation structurée, indispensable pour traiter des volumes de données variables sans dupliquer de code. Il détaille les critères de choix entre boucles bornées (POUR) et non bornées (TANT QUE, RÉPÉTER... JUSQU'À), tout en apportant une rigueur scientifique à travers les outils formels de preuve : le variant de boucle pour prouver la terminaison (absence de boucle infinie) et l'invariant de boucle pour garantir la correction absolue du résultat calculé.

## 1. Les Trois Structures d'Itération et le Choix Algorithmique

L'itération consiste à répéter un bloc d'instructions tant qu'une condition reste vérifiée ou pour un nombre de fois déterminé. La boucle 'POUR' gère elle-même son compteur de bornes. À l'inverse, dans les boucles 'TANT QUE' et 'RÉPÉTER... JUSQU'À', c'est au développeur de faire évoluer explicitement l'état des variables dans le corps sous peine de créer un blocage. Le choix d'une boucle dépend de deux questions : connaît-on le nombre de tours à l'avance ? Et le traitement doit-il s'exécuter obligatoirement au moins une fois ?

- Anti-pattern du Copier-Coller : Dupliquer des lignes de code rend les programmes rigides, illisibles et inadaptés aux flux de données dynamiques.
- Itération Bornée (POUR) : Structure employée lorsque le nombre d'itérations est fixe et connu au préalable. La variable de contrôle s'incrémente automatiquement.
- Itération Non Bornée à Pré-condition (TANT QUE) : La condition est évaluée avant le corps de boucle. Le traitement peut être exécuté 0 fois si la condition est fausse dès le début.
- Itération Non Bornée à Post-condition (RÉPÉTER... JUSQU'À) : Le corps de boucle est exécuté au moins une fois avant l'évaluation de la condition d'arrêt (1 à N exécutions).

***Cas concret enseignant :*** *L'analogie du coureur sur la piste d'athlétisme : Si l'entraîneur dit 'Fais 10 tours', c'est une boucle POUR. S'il dit 'Cours tant qu'il ne pleut pas', c'est un TANT QUE (s'il pleut déjà au vestiaire, il ne court pas du tout = 0 tour). S'il dit 'Fais un tour et arrête-toi dès que tu es essoufflé', c'est un RÉPÉTER... JUSQU'À (le coureur fera au moins un tour avant d'évaluer son état).*

## 2. Preuve Formelle de Terminaison : Le Variant de Boucle

Pour démontrer formellement qu'une boucle s'arrête en un temps fini, il faut exhiber un variant de boucle. C'est une fonction des variables du programme qui produit un entier naturel. Puisqu'il est impossible d'avoir une suite infinie d'entiers strictement décroissants et inférieurs à une constante plancher (propriété du bon ordre des entiers naturels), l'existence de ce variant prouve scientifiquement la terminaison de la boucle.

- Boucle Infinie : Dysfonctionnement majeur où la condition de sortie n'est jamais atteinte, provoquant un gel du programme.
- Variant de Boucle : Grandeur entière positive V associant un nombre à l'état courant de l'algorithme.
- Propriété de Décroissance : Le variant doit décroître strictement d'au moins 1 à chaque itération ($V_{k+1} < V_k$).
- Minorant : Le variant est borné inférieurement (généralement par 0), ce qui impose une limite stricte au nombre de tours.

***Cas concret enseignant :*** *L'analogie du réservoir de voiture : À chaque kilomètre parcouru (itération), le volume d'essence dans le réservoir (variant) diminue obligatoirement. Comme le réservoir ne peut pas descendre sous 0 litre (minorant), la voiture finira obligatoirement par s'arrêter. Si la jauge ne baisse pas, la voiture roulerait à l'infini : c'est l'équivalent du bug de la boucle infinie.*

## 3. Preuve Formelle de Correction : L'Invariant de Boucle

Garantir qu'une boucle s'arrête ne suffit pas, il faut prouver qu'elle produit le résultat exact. L'invariant de boucle est une méthode de démonstration basée sur la récurrence. Lorsque la condition d'arrêt est satisfaite, l'invariant est toujours préservé ; sa conjonction avec la négation de la condition de boucle démontre mathématiquement la validité du résultat final.

- Invariant de Boucle : Assertion (propriété logique) qui reste VRAIE avant, pendant et après chaque itération.
- Initialisation : Preuve que l'invariant est vrai juste avant le premier tour de boucle.
- Conservation (Hérédité) : Démonstration que si l'invariant est vrai au début d'une itération k, il demeure vrai à la fin de cette itération k.
- Conclusion (Correction) : Combinaison logique de l'invariant et de l'arrêt de la boucle garantissant que le résultat final est exactement celui recherché.

***Cas concret enseignant :*** *L'exemple du sac du livreur de journaux : Au départ, 'Journaux distribués + Journaux restants dans le sac = Total initial'. À chaque maison servie, les deux quantités varient, mais leur somme reste rigoureusement invariante. Quand le sac est vide à la fin de la tournée, le nombre de journaux distribués correspond exactement au total initial.*

## ■ Questions & Interventions des Étudiants en Classe

**Question étudiant :** Est-ce qu'on peut systématiquement remplacer une boucle POUR par une boucle TANT QUE ?
**Réponse du professeur :** Oui, algorithmiquement, toute boucle POUR peut s'écrire avec un TANT QUE en initialisant le compteur avant et en gérant son incrémentation manuellement dans le corps. Cependant, si le nombre de tours est connu à l'avance, la boucle POUR est fortement recommandée car elle rend l'intention claire et évite d'oublier l'incrémentation.

**Question étudiant :** Si j'utilise une boucle TANT QUE avec la condition (i < N), est-ce que le corps s'exécute si i est déjà égal à N dès le début ?
**Réponse du professeur :** Non. La boucle TANT QUE est à pré-condition : l'évaluation a lieu AVANT de pénétrer dans le bloc d'instructions. Si i vaut N dès l'entrée, la condition (i < N) est fausse immédiatement, le corps est complètement ignoré et exécuté 0 fois.

## ■ Exercice & Démonstration d'Application

**Énoncé :** Écrire en pseudo-code l'algorithme permettant de calculer la somme des N premiers entiers pairs stricts (ex: pour N=3, sommer 2 + 4 + 6 = 12). Justifier le choix de la structure répétitive et prouver la terminaison à l'aide d'un variant.

**Résolution pas à pas :**
1. Choix de la boucle : Le paramètre N est fixé dès l'entrée, le nombre d'itérations est connu d'avance. La boucle POUR est la plus adaptée. 2. Initialisation des variables : 'somme' reçoit 0 (accumulateur), 'terme' reçoit 2 (premier entier pair strict). 3. Pseudo-code : Algorithme SommePairs Variables : N, i, somme, terme : Entiers Début Lire(N) somme <- 0 terme <- 2 Pour i allant de 1 à N Par pas de 1 Faire somme <- somme + terme terme <- terme + 2 FinPour Afficher(somme) Fin 4. Démonstration du variant : Si exprimé avec une boucle tant que sur le compteur i, la quantité V = (N - i + 1) est strictement décroissante de 1 à chaque tour et minorée par 0. Elle s'annule quand i dépassera N, ce qui prouve la terminaison.

**Résultat : Pour N = 3 : - Tour 1 (i=1) : somme = 0 + 2 = 2, terme devient 4. - Tour 2 (i=2) : somme = 2 + 4 = 6, terme devient 6. - Tour 3 (i=3) : somme = 6 + 6 = 12, terme devient 8. Sortie de boucle. Affichage : 12. L'algorithme se termine et produit le résultat exact.**

## ■ Auto-Évaluation Flash d'Examen

**Q : Quelle est la différence majeure entre une boucle TANT QUE et une boucle RÉPÉTER... JUSQU'À ?**

*R : La boucle TANT QUE teste la condition en entrée (0 à N exécutions), tandis que RÉPÉTER... JUSQU'À teste la condition en sortie, garantissant au moins une exécution (1 à N).*

**Q : Quelles sont les deux conditions obligatoires qu'une quantité doit remplir pour constituer un variant de boucle valide ?**

*R : Elle doit être une valeur entière strictement décroissante à chaque itération et être minorée par une constante (généralement 0).*

**Q : Quel oubli classique dans une boucle TANT QUE fait perdre des points en DS d'algorithmique ?**

*R : Oublier de modifier la variable de contrôle dans le corps de la boucle (ex: oublier i <- i + 1), ce qui génère une boucle infinie.*

*Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765*